RetailStore_2n

How it works

A secondhand marketplace, explained from the browser down to the database — what each piece does, why it was built that way, and which numbers were measured rather than claimed.

ARCHITECTURE REFERENCE 32 DECISION RECORDS

35 TEST SUITES **1,263 ASSERTIONS PASSING**

THE IDEA

A marketplace where the damage is part of the description

RetailStore_2n is a marketplace for things that have already had one life, where an item's wear and repairs are part of its story, not something to hide.

The site sells secondhand furniture, clothing and objects, and it takes that idea literally: **condition is disclosed, not glossed over.** A chair with a repaired leg says so. Almost every item is one of a kind — one chair, one jacket, one mended plate — and that single fact shapes more of the engineering than anything else on this page.

Because if there is only one chair, the hard problem is not counting stock. It is making absolutely certain that two people who click "buy" at the same instant cannot both get it. That constraint comes back again and again below, in slightly different clothes: two buyers and one chair, two refunds and one order, two workers and one notification.

What you can actually do on it

Browse and search a catalogue, save things to a wishlist, buy with a card, get refunded when a seller can't send an item, leave a review if

you actually bought the thing. As a seller: list items with photos, verify your identity, mark orders shipped. As a moderator: see everything, remove listings, suspend accounts, and answer "did that customer get their email?"

Money now completes its whole circuit. A buyer pays the platform; the platform pays the seller once the item has been delivered and a return window has closed; and a buyer who receives something broken can **ask for their money back** from the order itself, which the seller answers and a moderator settles if they disagree.

That last sentence was the largest gap in this document for most of the project's life, and it is worth saying why it took so long. The refund machinery existed early — per-line, guarded against over-refunding, settled by webhook. What was missing was never the plumbing. It was deciding *who is allowed to say yes*, which is a product question wearing an engineering costume.

THE SHAPE

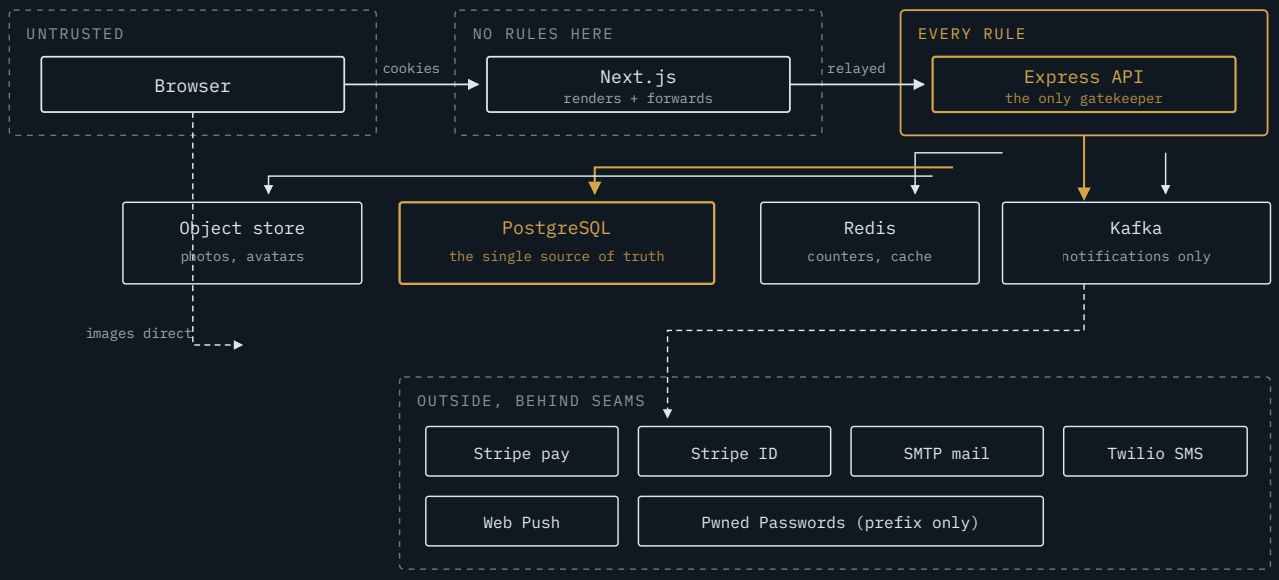
Six moving parts, and only one of them is allowed to say no

There are two Node programs, not one, and the split is the most important structural decision in the system. The browser never talks to the API directly. It talks to the Next.js server, which forwards the request onward without interpreting it.

That forwarding server is called a **BFF** — a backend for the frontend. It holds no business rules at all. Every rule lives in the Express API, which means someone who bypasses the website entirely and pokes the API with a script gains nothing: the same checks run either way. A greyed-out button is a courtesy; the server re-checks.

PLATE 01

What runs, and what each part is trusted with



Dashed boxes are zones of trust; the dashed arrow from the browser is the one request that skips the API entirely — photos are fetched straight from the object store. **Gold marks Postgres and the API**: the only two things the system treats as authoritative. Redis and Kafka can be wiped without losing an order, a notification or a penny.

> THE RULE THIS BUYS
>
> Redis and Kafka are allowed to lose everything they hold. Rate-limit counters can evaporate; the cache can be cold; the broker can be empty. None of that loses an order, a notification, or money — because the record of all three lives in Postgres, and the other two are only ever carrying *work in progress*.

A REQUEST

What happens between the click and the answer

Signing in gives the browser two cookies, and they do very different jobs. Neither can be read by JavaScript, so a script injected into the page cannot steal a session.

THE TWO COOKIES

COOKIE	WHAT IT IS	LASTS	STORED IN THE DATABASE AS
access	A signed token the API can verify without a database lookup	15 min	Not stored at all — it is self-proving
refresh	A long random string, good only for getting a new access token	30 days	A **SHA-256 hash**, never the value

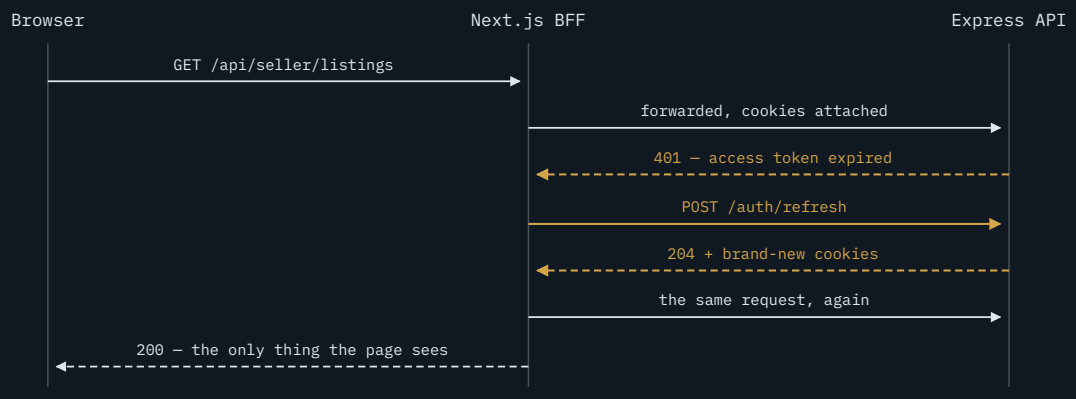
Hashing the refresh token matters for the same reason hashing a password does: if the database leaks, an attacker gets hashes, and a hash cannot be presented as a cookie. That refresh token is every bit as good as a password for impersonating someone, so it gets the same treatment.

The renewal nobody sees

Access tokens expire every fifteen minutes, which would mean being logged out constantly. Instead the BFF handles it: when the API says "expired", the BFF quietly renews and retries the original request. The browser only ever sees the successful answer.

PLATE 02

An expired token, renewed without the user noticing



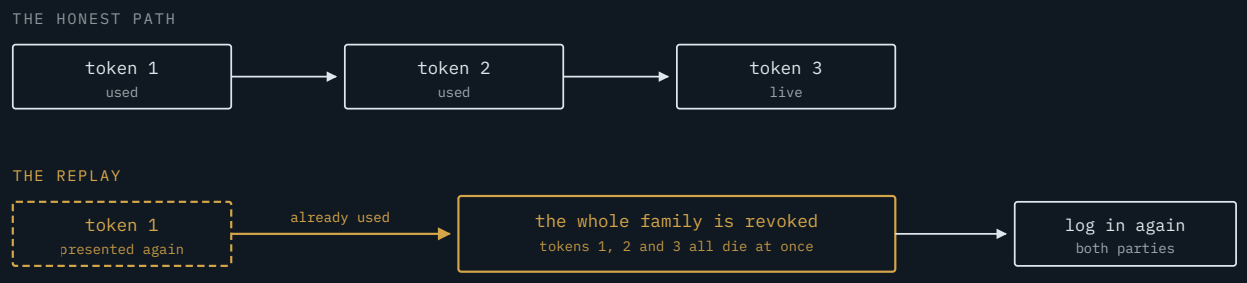
Solid lines are requests, dashed are responses. **The gold detour is invisible to the page.** Login, signup and refresh itself are excluded from this retry — otherwise a wrong password would trigger an endless renewal loop instead of an error message.

What happens if a refresh token is stolen

Every renewal issues a new refresh token and marks the old one used. So if a token that has *already been used* shows up again, only one thing can explain it: a copy is in someone else's hands. The system does not try to guess whose — it kills the entire session family. The real user logs in again. The thief gets nothing that lasts.

PLATE 03

Rotation, and how a stolen token gives itself away



Reuse is the signal. A token that works exactly once means a replay cannot be mistaken for normal traffic, so no judgement call is needed about whether a leak happened. The cost is real and accepted: a legitimate user with a flaky connection can occasionally be logged out.

THE HARD PROBLEM

Two buyers, one chair

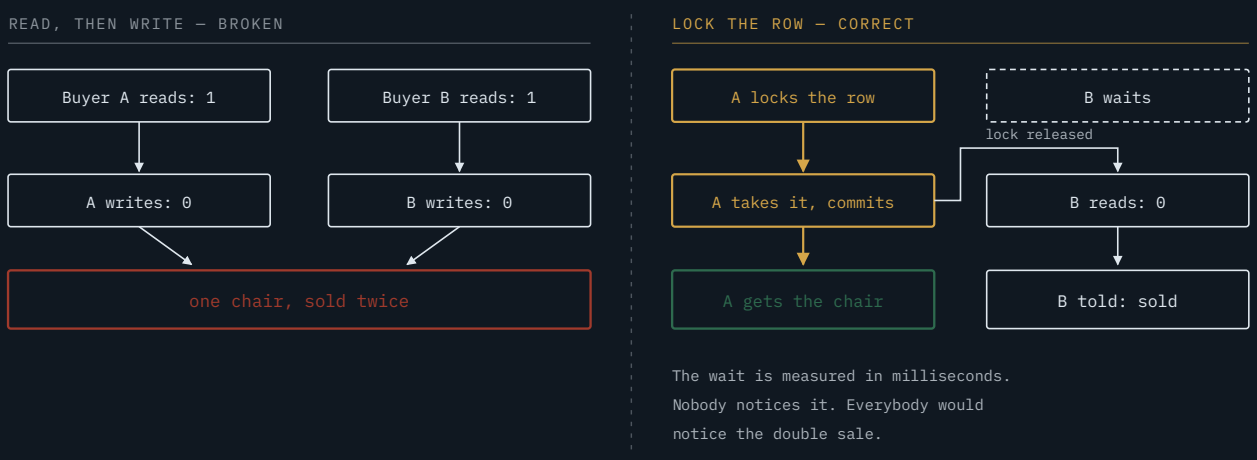
Here is the bug that a marketplace of unique objects must never have. Checking stock and then taking it looks obviously correct and is obviously wrong:

- Two requests arrive at the same moment.
- Both read the chair. Both see quantity 1.
- Both decide they may proceed.
- Both write quantity 0. The chair sells twice.

The window is tiny — a few milliseconds — which is exactly why it survives testing and appears in production. The fix is to make the database itself hand out the answer, one caller at a time.

PLATE 04

The same two clicks, before and after the lock



One figure, one claim: the only difference between the halves is who decides. On the left the application decides, from a value it read a moment ago. On the right **Postgres decides**, and the second caller simply cannot see stale stock.

The same shape, five more times

Once you can see this pattern you can see it everywhere in the codebase, and that repetition is deliberate — one idea learned once, applied wherever it fits. The two most recent features in the system, seller payouts and buyer returns, needed no new idea at all: both are the chair again, in different clothes.

ONE DECISION, MADE ATOMICALLY, BEFORE ANY OUTSIDE CALL

WHERE	WHAT COULD GO WRONG TWICE	HOW IT IS CLAIMED
Checkout	Two buyers get one chair	`SELECT … FOR UPDATE`
Payment	One order charged twice	Conditional UPDATE before calling Stripe
Refunds	More refunded than was paid	Refund headroom claimed the same way
Notifications	One event, two emails	A unique row inserted before the provider is called
Payouts	One sale paid to a seller twice	A unique row per order line, inserted before the transfer
Returns	Two requests on one item, answered separately	A unique row per order line, and the insert *is* the request

> THE RULE, STATED ONCE
>
> **One system decides, atomically, and the outside call happens only after that decision is durable.** Never the other way round — because if you call Stripe first and then record it, a crash in between leaves money moved and no record of it.

TELLING PEOPLE THINGS

The notification pipeline, which is most of the machinery

Eleven things can happen that a person should hear about: an item sold, an order shipped, a refund issued, an account suspended. Each

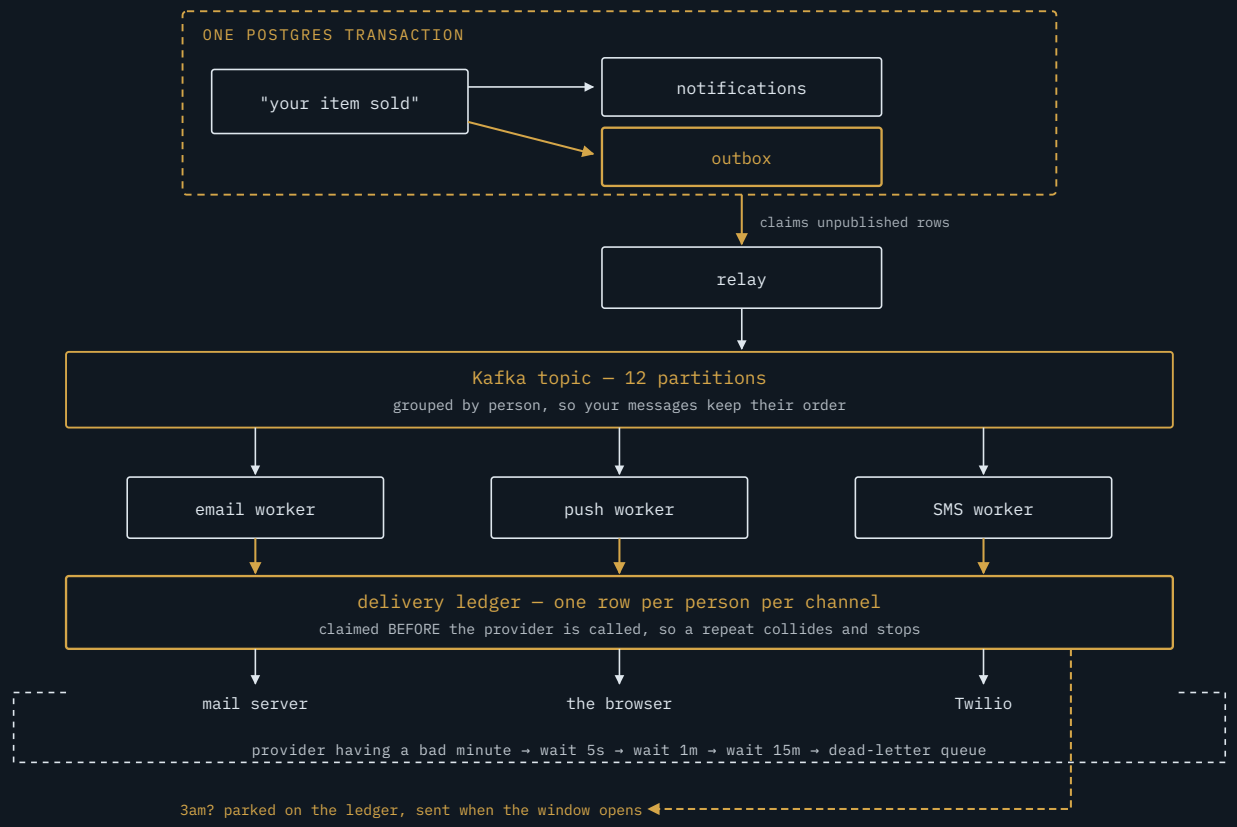
one has to reach the right person, on the channels they agreed to, exactly once.

The naive version is to save the notification and then send the email. That is **two writes to two systems with no transaction around them**, and a crash in the gap is silent: either a notification nobody was told about, or an email about something that got rolled back. Nothing anywhere records that the pair was supposed to be atomic.

> **THE SEAM THIS CLOSES**
>
> So the event is written into an **outbox** — an ordinary table — in the very same transaction as the notification. They commit together or not at all. A separate small program, the **relay**, reads committed outbox rows and publishes them onward. A crash anywhere now leaves work *visibly unfinished* rather than quietly lost.

PLATE 05

From one transaction to three channels, and what happens when a send fails



Gold marks every place something is claimed or made atomic — the shared transaction, the outbox, the topic key, the ledger. The dashed loop at the bottom is the failure path: a message that keeps failing waits somewhere else rather than blocking everyone behind it.

Why waiting happens on a separate queue

When a send fails for a reason that might fix itself — a timeout, a server having a bad minute — the obvious move is to wait and try again. The obvious move is wrong. A worker that sits waiting holds up every other message queued behind it, so **one person with a dead phone number would delay everybody else's notifications.**

Instead the message is pushed onto a separate queue with a built-in delay, the worker moves on, and something else picks it up later. Three rungs — five seconds, one minute, fifteen minutes — and then it lands in a dead-letter queue where a human can look at it.

Failures that *cannot* fix themselves never enter that ladder at all. A rejected address, a deleted account, or a recipient who replied STOP

is permanent. Retrying those is how a sender's reputation gets burned, and in the case of an opt-out it is also unlawful.

Consent, before anything is sent

Each of the eleven events has a default for each channel, and each person can override it. Two of them default to SMS — a refund, and an order that cannot be sent — because in both cases money moved and the person needs to know away from a screen. A review notification does not clear that bar.

- **Nothing is texted to an unverified number.** A six-digit code, valid ten minutes, five guesses, then it is spent.
- **A parcel's delivery phone is never reused for messaging.** It is often a third party — a gift, a workplace, a relative — who never agreed to anything and cannot unsubscribe.
- **Quiet hours defer rather than drop.** A message caught at 3am is parked and sent when the window opens, once, by exactly one server.
- A daily cap per person, as a circuit breaker against bugs rather than as a preference.

THE NUMBERS

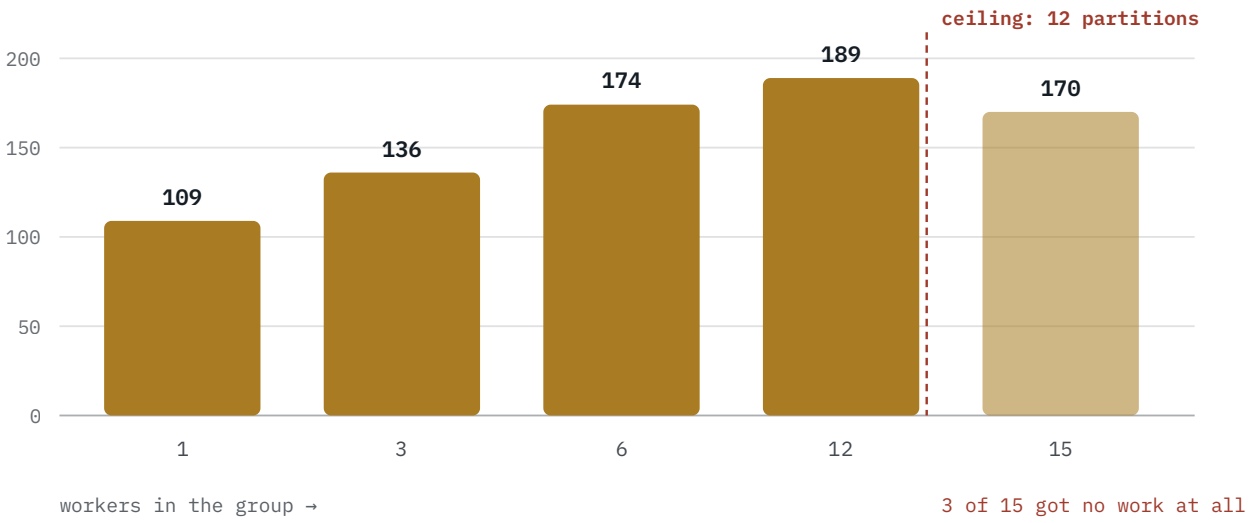
Measured, including the unflattering parts

This project has a rule about performance: **measure it, don't claim it.** Five scripts exist to make the claims checkable, and the database-transaction counts come from Postgres's own counters rather than from the application, because an application counter measures what the code *believes* it did.

The notification pipeline was built partly to scale. So the honest question is: does it? The answer is more interesting than a yes.

Adding workers helps, then stops helping

10,000 NOTIFICATIONS THROUGH THE REAL EMAIL PATH · EVENTS PER SECOND



Twelve times the workers buys **1.7×** the throughput, and past twelve it gets worse. The topic has twelve partitions, so the thirteenth worker onward is given nothing — that figure came from asking the broker which worker owned which partition, not from guessing at the shape of the curve, because a slow machine produces the same curve for a completely different reason.

And the queue was never the slow part

SAME RUN · EVENTS PER SECOND, ONE LINEAR SCALE



The broker keeps up roughly **eighteen times** more easily than the workers do. What is actually slow is the per-message database work inside each worker — which means the "we need a queue for throughput" argument is the weakest reason this pipeline exists. It earns its place on fanning one event out to three channels, on keeping a dead SMS provider from stalling email, and on being able to replay a day's messages after fixing a bug.

The rest of the measurements

Each of these comes from a script in the repository that can be re-run, and each one shows the thing failing before it shows it working — a number with no failing baseline next to it is hard to trust.

928 → 0 DATABASE QUERIES, COLD VS CACHED	**30 → 10** LOGIN ATTEMPTS PAST A LIMIT OF 10, BEFORE AND AFTER SHARING THE COUNTER	**11.1 ms** MEDIAN LAG OF THE STANDBY DATABASE	**500 rows** RECOVERED AFTER A TABLE WAS DELIBERATELY DROPPED
0 and 0 DUPLICATES AND LOSSES WHEN A WORKER WAS KILLED MID-BATCH	**1,263** ASSERTIONS ACROSS 35 SUITES, NO MOCKS		

THE ONE WORTH PAUSING ON

A worker was killed outright — no warning, no cleanup — with **2,215 of 3,000 messages still outstanding**. The survivors finished all 3,000, with zero duplicates and zero losses. Two mechanisms had to hold at once for that: the position marker is only advanced *after* a message is handled, so a crash re-delivers rather than skips; and the ledger refuses the second claim, so a re-delivery cannot become a second email. Either one alone fails — advance-first loses messages, no-ledger doubles them.

SWAPPABLE PARTS

Every outside service sits behind a seam

Seven things live outside: taking payments, paying sellers, checking identity documents, sending email, sending texts, storing files, and carrying notifications between processes. Each one is reached through a single small module with two implementations behind it — the real service, and a stub — chosen by one environment variable.

The stubs are not placeholders for unwritten code. They are what the automated tests run against, which is why the whole purchase, refund and verification flow can be exercised with no credentials at all — and why someone contributing from a fork of the repository, who cannot read its secrets, still gets a meaningful green test run.

SEVEN SEAMS

CONCERN	THE REAL ONE	THE STUB	CHOSEN BY
Payments	Stripe, in test mode	In-process, driven by test card numbers	`PAYMENT_PROVIDER`
Identity	Stripe Identity	Outcome decided by the document number	`KYC_PROVIDER`
Email	Any SMTP server	Printed to the terminal, or a throwaway inbox	`MAIL_TRANSPORT`
SMS	Twilio, over its REST API	Recorded in memory; returns the code so a person can finish the flow	`SMS_PROVIDER`
Notifications	Kafka, with separate worker processes	Handed straight to the same functions, in-process	`NOTIFY_TRANSPORT`
File storage	Any S3-compatible object store	Local disk, with the same key validation so a swap cannot orphan a URL	`STORAGE_DRIVER`
Payouts	Stripe Connect: accounts, onboarding, transfers, reversals	Accounts in memory and a transfer id derived from the payout, so the whole path runs with no Stripe account	`PAYOUT_PROVIDER`

The API prints which of each it is using when it starts, and the interface says so on screen where a user could be misled. **A stub that silently looks real is worse than no stub at all.**

Identity documents are the sharpest case

A government ID is exactly the kind of data that turns a database into something worth attacking, for no product benefit. So the system stores a reference to somebody else's verification result and never the document — and with the real provider the document never reaches

this server in the first place. Switching provider does not merely swap an implementation; it shrinks what the application is *capable* of leaking.

THE DATA

Twenty-eight tables, in six groups

Postgres is the only thing that is ever authoritative. Everything else in the system can be rebuilt from it.

WHAT IS STORED, AND THE ONE RULE THAT GOVERNS EACH GROUP

GROUP	TABLES	THE RULE THAT MATTERS
People	`users` · `refresh_tokens` · `email_verification_tokens` · `password_reset_tokens` · `phone_verifications`	Every credential is hashed. One account type — selling is a capability you switch on, not a different kind of user
Selling	`seller_profiles` · `kyc_attempts` · `categories` · `listings` · `listing_images`	Listings are never really deleted — a past order has to stay readable, so they are marked removed instead
Buying	`reservations` · `orders` · `order_items` · `addresses` · `wishlist_items`	Money is stored as whole cents, never a decimal, so no rounding can happen in transit. Addresses are copied onto the order, so editing your address later cannot rewrite history
Trust	`reviews` · `reports` · `moderation_actions` · `refunds`	Ratings are computed from review rows rather than stored, and the moderation log can only be appended to — including by whoever wrote it
Telling people	`notifications` · `outbox_events` · `notification_deliveries` · `notification_preferences` · `push_subscriptions`	The notification is the record; a delivery is one attempt on one channel. Preferences store only what someone actively changed
`payouts` · `payout_items` · `payout_debts` · `return_requests`	Money going back out, and the two constraints that make it safe: one payout line per item sold, and one return request per item, both enforced by the database rather than by a check in the code	

Two details that explain a lot of the rest

Preferences store deviations, not state. An absent row means "use the default". The alternative — a row per person per event per

channel, created at signup — would mean that adding a twelfth event type requires touching every account in the database before it can be sent at all, and would store thirty-three rows per person to record that they never opened the settings page.

A delivery record outlives the account it was for. There is deliberately no foreign key from a delivery to a user, so "this was sent to an account since deleted" remains a real, answerable question.

CONFIDENCE

How it is tested, and what testing means here

Thirty-five suites, 1,263 assertions, and **nothing is mocked.** The tests drive a real Postgres, the real Express API, and a production build of the site under a real browser — on the view that the bugs worth catching live in the seams between the pieces rather than inside any one of them.

That has a cost, and it is paid deliberately: the suite needs a database, a Redis, and both servers running, and it takes about thirteen minutes.

Testing a message broker without requiring one

Adding Kafka to every test run would slow the pipeline and give contributors one more thing to install. So the notification transport has the same shape as the payment and identity seams: by default the relay hands events *straight to the same worker functions*, in-process, no broker involved. Preference resolution, the duplicate guard, template rendering, the provider calls — all of it runs for real. Only the wire is absent.

The handful of claims that genuinely need a broker live in one gated section, and when there is no broker that section **skips loudly and names what went unproven** rather than passing quietly. A section that reports green for something nobody ran is worse than no section, because it gets believed.

FOUND BY FINALLY RUNNING IT

That gated section had never actually executed. The first time it did, it *failed* — it produced its messages after subscribing and lost a race with the broker's group assignment, which no amount of waiting would have fixed reliably. Gated code that has never run is not tested; it is only written.

RUNNING IT

One image definition, several programs

A single Dockerfile describes every process the system can run, each as its own build target: the API, the website, the database with its backup tooling, the notification relay, and the channel workers. The relay and the workers are separate programs precisely so they can be scaled and can fail independently of the website.

Bringing everything up is one command. Two of the containers exit immediately, and that is correct — one applies pending database migrations and one creates the file-storage bucket, both finishing before the API starts. Doing either of those at API startup is the usual shortcut, and it breaks the moment there is more than one copy of the API racing to do it.

Optional pieces sit behind named profiles, so a plain start-up stays small: the standby database, the message broker with its relay and workers, and one-off tools like seeding and backups.

And on a cluster, where the database can be replaced

Compose can start everything on one machine and it can do nothing about a machine that dies. So the same images also have Kubernetes manifests, and the interesting part is what happens to Postgres: it is not a workload written by hand here but a **CloudNativePG cluster**, which is an operator whose whole job is knowing how to promote a replica when the primary stops answering.

That claim was tested rather than assumed. Deleting the primary's pod on a local cluster produced a promoted replica, and a write submitted afterwards — a real signup, through the website, to the database — succeeded. The figure worth noticing is the last one:

2 of 2 ROWS FOUND IN THE PROMOTED INSTANCE: ONE WRITTEN BEFORE THE KILL, ONE AFTER	**0** API RESTARTS NEEDED TO RECONNECT
199s PROMOTION, ON ONE LAPTOP NODE WITH NO SPARE MEMORY	

> **WHY THE RESTART COUNT IS THE NUMBER TO READ**
>
> Any orchestrator will restart a pod, and a restart would have hidden the question. The connection pool inside the API had cached an address that stopped being the primary — and it recovered on its own, because it connects to a name the operator moves rather than to an instance. Had it needed bouncing, the demonstration would still have "passed" while concealing the thing worth demonstrating.

Two things are honest to say about it. **199 seconds is slow**, and it is printed rather than smoothed over: a promotion is a short automatic outage, not an invisible one, and the site did return errors during the window. And **this ran on a laptop, not a cloud.** A manifest that applies locally can still be wrong about a load balancer.

One scheduled job, and five that stayed where they were

A cluster brings a scheduler, and the obvious move is to take every background loop in the application and make each one a scheduled job. That would have been a step backwards. Five recovery sweepers run inside the API — returning expired holds to stock, settling payments whose request died, sending messages that quiet hours parked — and each of them claims its work with the same conditional write the rest of this document is about. Which means *several copies divide the work* rather than repeating it, and one scheduled job would have serialised what currently parallelises.

Exactly one thing became a scheduled job: finishing a payout that was claimed and never transferred, because the provider timed out. It is scheduled for a reason that has nothing to do with tidiness — it

moves money to somebody else. A job has an exit code, a start time and a record; a loop inside the process serving checkout has only log lines, and shares a thread pool with people trying to buy things.

HONESTY

What is deliberately not built

A description that only lists what works is a sales document. These are the real edges, and most of them are known trade-offs rather than oversights.

KNOWN GAPS

GAP	WHY IT IS STILL OPEN
No real money has ever moved	The largest one, and it did not shrink when payouts were built — it moved. Payments, identity and payouts all run against provider stubs or Stripe's test mode. No connected account has been created, no transfer issued, no document checked. What is proven is the ordering and the constraints; what is unproven is every provider behind them
No cluster but a local one	Kubernetes manifests exist and were verified on kind: deleting the Postgres primary promoted a replica and a write then succeeded with zero API restarts. But a manifest that applies locally can still be wrong about a cloud load balancer, an ingress class or a storage class — and the Kafka half of the deployment is not written at all
A return is approved on trust	Nothing tracks the parcel going back. Approval is the seller saying they are content, because a scan-triggered refund needs a carrier integration that does not exist — the same one that made prepaid outbound labels a promise worth deleting rather than keeping
Search reads every row	It is substring matching, which no index can help with. Fine at this size; a proper text index is the upgrade
Backups are not scheduled	Still true in both deployments. Recovery works and has been rehearsed, but a base backup is taken by hand and nothing expires old ones. CloudNativePG does scheduling, retention and verification in one block — and the manifests deliberately leave it out, because it needs a bucket and credentials that differ per environment. The mechanism has a home and nothing uses it

GAP	WHY IT IS STILL OPEN
Quiet hours use the server's clock	Deferring works; the timezone is wrong. There is no timezone recorded against a person, so someone abroad is quiet during the server's night rather than their own
Nothing reads the standby	On purpose. Sending reads there introduces a class of bug where you cannot see your own order right after placing it, and caching already absorbed the read volume
One test assertion is flaky	A card-data check scans every order for the test cards' last four digits, which can match an unrelated identifier another suite left behind. It is a scoping problem in the check, not a leak

> AND ONE THAT IS WORTH NAMING AS A PATTERN
>
> Three pieces of the notification pipeline had to be built *after* the phase that argued for them was declared finished: the retry ladder was designed and never wired up; quiet hours shipped as a silent drop rather than a deferral; and the sweeper that rescues a half-finished send went unbuilt through four phases. Each was caught by writing something that measured the claim rather than restating it.

THE REASONING

Thirty-two decisions, each with its rejected alternatives

Every non-obvious choice above has a record explaining what was decided, what else was considered, and what it costs. They are load-bearing documents: they carry the reasoning the code cannot.

0001	Access and rotating refresh tokens, with reuse detection	0002	Next.js as a forwarding layer, not direct browser-to-API calls
0003	One account type; selling is a capability	0004	Password length and a breach check, not composition rules

0005	Money as whole cents	0006	Store a verification reference, never the document
0007	Verification gates payouts, not listing	0008	Soft delete plus a status, not a stock count
0009	Compute ratings from review rows	0010	Re-encode uploads to strip location data
0011	Avatars in object storage, referenced by URL	0012	Row-level locking for stock
0013	Claim the payment before charging it	0014	One order per basket, fulfilment per line, addresses copied
0015	Admin granted only from a shell, then a second factor to open the panel	0016	Claim refund headroom atomically, then refund
0017	Password change and reset without revealing who has an account	0018	Redis for shared throwaway state only, never a source of truth
0019	A rule for what may be cached — which amends 0018	0020	A streaming standby, and recovery to any point in time
0021	Concurrent token refresh settled in Postgres, not with a lock	0022	Object storage for uploads — the last thing blocking a second server
0023	One Dockerfile with a target per program	0024	An outbox, instead of writing to two systems and hoping
0025	One topic keyed by person; delays on separate queues	0026	A delivery ledger, claimed before the provider is called
0027	Consent before delivery: verified numbers, per-event choices, unsubscribe	0028	Twilio for SMS, behind the same kind of seam as payments
0029	Pay sellers with separate transfers, not destination charges	0030	What makes money payable, and the ledger that stops it being paid twice
0031	Buyer-initiated returns, as a request the seller answers	0032	An operator for Postgres, plain manifests for everything else

VOCABULARY

The handful of words that do the most work

Seam

A single small module that stands between the system and an outside service, with a real implementation and a stub behind it. Changing provider means editing one file.

Outbox

An ordinary database table that events are written into, in the same transaction as the thing they describe, so the pair cannot come apart.

Relay

The small program that reads committed outbox rows and publishes them onward. Several can run at once without publishing anything twice.

Ledger

One row per notification per channel, written *before* the provider is called. A repeat attempt collides with that row and stops, which is what makes an at-least-once queue safe to point at a phone.

Claim

A single write whose success *is* the permission to proceed. Used instead of checking and then acting, because two callers can both pass a check and only one can win a write.

Suppressed

A delivery that was deliberately not sent — the channel was off, there was no verified number, it was quiet hours. Recorded rather than dropped, so "why did nothing arrive?" has an answer.

Sweeper

A timer that recovers state nothing in the normal request path will ever reach again: expired holds, unpaid orders, parked messages, half- finished sends, spent outbox rows.

```
RetailStore_2n · architecture reference
Sources of record: docs/architecture/, docs/adr/ (28
records), docs/plans/0001
```

Every figure on this page came from a script in the repository that can be re-run.